

Securitatea aplicației workout

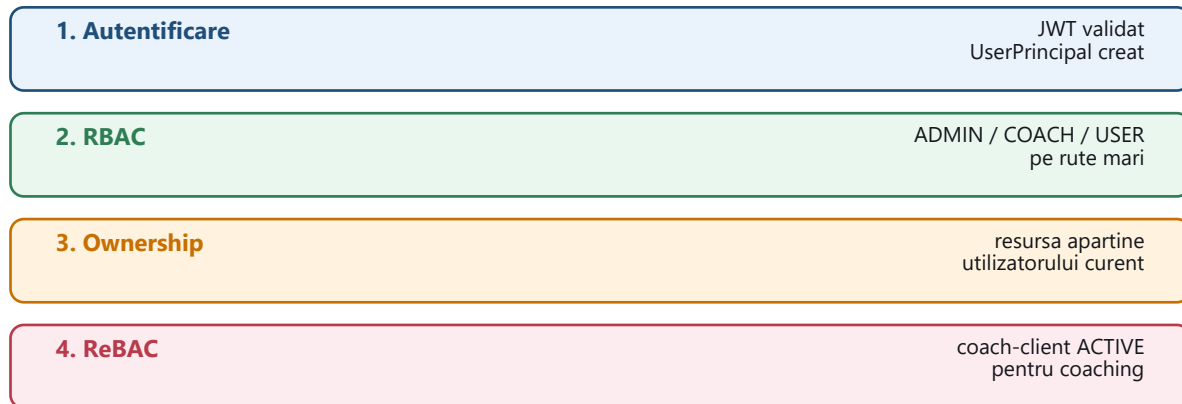
Ghid de învățat pentru prezentare - autentificare, autorizare, IDOR, RBAC și ReBAC

Acest PDF explică partea de securitate a proiectului exact pe implementarea din cod. Ideea centrală este că aplicația nu verifică doar dacă utilizatorul este logat, ci verifică și dacă acel utilizator are dreptul concret să acceseze resursa cerută.

Componentă	Implementare în proiect	Rol
Autentificare	JWT access token + refresh token opac	Identifică utilizatorul curent
Stocare parole	BCryptPasswordEncoder	Parolele nu sunt salvate în clar
Refresh token	Cookie HttpOnly + SHA-256 fingerprint în DB	Reînnoiește sesiunea fără login repetat
RBAC	SecurityConfig: /api/admin/**, /api/coach/**	Controlează zone mari ale aplicației
Ownership / IDOR	TemplateAccess, WorkoutSessionAccess, repository-uri owner-scoped	Blochează accesul la resursele altui user
ReBAC	CoachAccess + coach_client_relationships ACTIVE	Permite acces coach doar la clienții reali

1. Harta generală a securității

Securitatea este pe mai multe straturi. Primul strat identifică utilizatorul, dar asta nu este suficient. După autentificare, fiecare operație trebuie să verifice rolul, proprietarul resursei sau relația coach-client.



Formula scurtă pentru prezentare:

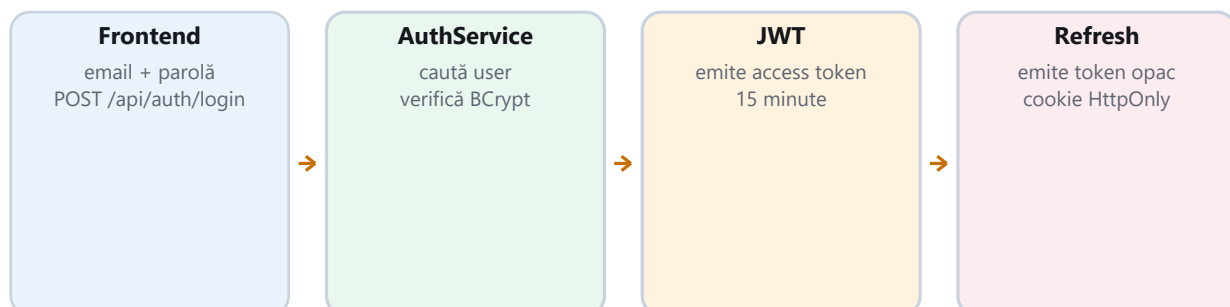
Autentificarea răspunde la întrebarea cine ești?. Autorizarea răspunde la întrebarea ai voie să accesezi această resursă? În aplicație, cele două sunt separate clar.

Zonele de cod relevante:

- SecurityConfig - definește rutele publice, rutele autentificate și rolurile cerute.
- JwtAuthenticationFilter - citește token-ul din header și populează SecurityContext.
- AuthService - login, register, refresh, logout, hashing parole și refresh tokens.
- TemplateAccess / WorkoutSessionAccess - ownership checks și protecție IDOR.
- CoachAccess - poarta ReBAC pentru relația activă coach-client.

2. Register și login

La register, backend-ul normalizează emailul, verifică dacă există deja, hash-uieste parola cu BCrypt și creează userul cu rolul USER. La login, backend-ul caută userul după email și verifică parola cu BCrypt.



Endpoint	Ce face	Rezultat
POST /api/auth/register	Creează cont nou, parola devine BCrypt hash	201 + access token + refresh cookie

Endpoint	Ce face	Rezultat
POST /api/auth/login	Verifică email și parolă	200 + access token + refresh cookie
GET /api/auth/me	Folosește UserPrincipal din JWT	Datele userului curent

Important: access token-ul este în body-ul răspunsului, dar refresh token-ul nu este expus în body. El este pus de backend într-un cookie HttpOnly.

3. Access token JWT

Access token-ul este JWT-ul folosit la cererile normale către API. Frontend-ul îl trimite în headerul Authorization. Backend-ul îl validează la fiecare request protejat.

Authorization: Bearer

Claims din JWT:

- subject = userId
- email
- role
- type = access
- issuer
- expiration

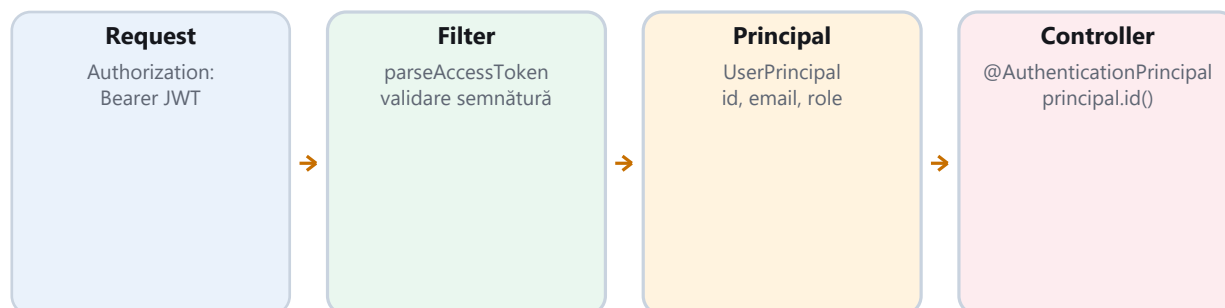
Ce verifică JwtService.parseAccessToken:

- semnătura token-ului, folosind cheia HS256;
- issuer-ul configurat;
- expirarea token-ului;
- claim-ul type = access, ca să nu fie acceptat alt tip de token;
- formatul claim-urilor: userId valid, rol valid.

JWT-ul nu este gândit să fie rotit la fiecare request. El este refolosit până expiră. În proiect, durata access token-ului este 15 minute.

4. Cum intră userul curent în backend

JwtAuthenticationFilter este filtrul care transformă token-ul din header într-un utilizator curent disponibil în cod.



```

@GetMapping("/{templateId}")
public TemplateDetailResponse get(@AuthenticationPrincipal UserPrincipal principal,
@PathVariable UUID templateId) {
return templateService.get(principal.id(), templateId);
  
```

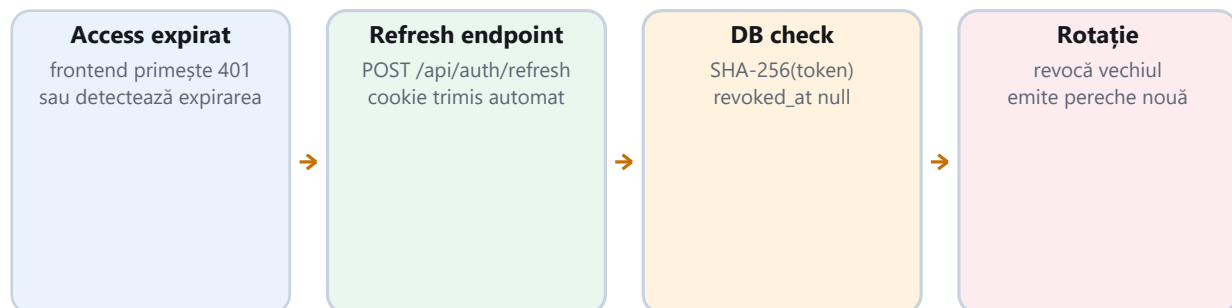
}

Asta este motivul pentru care frontend-ul nu trebuie să trimită userId pentru operațiile personale. Identitatea vine din JWT-ul validat pe backend, nu din parametrii trimiși de client.

5. Refresh token: ce este și când se schimbă

Refresh token-ul este folosit doar ca să obțină o pereche nouă de token-uri când access token-ul expiră. El nu este trimis la fiecare operație normală. Cererile normale folosesc același access token până la expirare.

Operație	Se schimbă access token?	Se schimbă refresh token?
GET /api/history	Nu	Nu
POST /api/workouts/start	Nu	Nu
POST /api/auth/login	Da	Da
POST /api/auth/register	Da	Da
POST /api/auth/refresh	Da	Da, token-ul vechi este revocat
POST /api/auth/logout	Nu emite token nou	Token-ul curent este revocat



Rotația înseamnă că refresh token-ul vechi este folosit o singură dată. După refresh, el este marcat revocat și nu mai poate fi reutilizat.

6. De ce refresh token-ul este opac și în cookie HttpOnly

Refresh token-ul din proiect nu este JWT. Este o valoare random, generată cu SecureRandom, de 32 bytes, encodată Base64 URL-safe. Se numește opac fiindcă nu conține informații citibile despre user.

Mecanism	În proiect	De ce ajută
Opac	Random 32 bytes, nu JWT	Nu expune userId, email sau rol
HttpOnly	Cookie creat de RefreshCookieFactory	JavaScript nu îl poate citi direct prin document.cookie
SameSite=Strict	Cookie trimis doar în context same-site	Reduce riscul de CSRF
Path=/api/auth	Cookie disponibil doar pe rutele auth	Nu circulă la toate endpoint-urile
Secure	false local, true în producție prin profil	Pe HTTPS, cookie-ul nu pleacă pe HTTP

Dacă un atacator controlează complet browserul sau calculatorul, orice mecanism poate fi compromis. Scopul aici este să reducem riscurile normale: XSS simplu, token-uri accesibile în JavaScript, leak-uri din DB.

7. SHA-256 fingerprint pentru refresh token

În baza de date nu se păstrează refresh token-ul brut. Se păstrează doar amprenta lui SHA-256 în tabelul `refresh_tokens`, coloana `token_hash`.

```
raw_refresh_token -> SHA-256 -> token_hash
```

```
refresh_tokens:
```

```
- id
- user_id
- token_hash
- expires_at
- revoked_at
- created_at
```

Avantajele acestei decizii:

- dacă baza de date este expusă, refresh token-urile brute nu pot fi folosite direct;
- backend-ul poate căuta token-ul primit calculând același hash;
- backend-ul poate revoca token-uri individuale prin `revoked_at`;
- rotația token-ului poate fi verificată în baza de date.

Diferență față de parolă: pentru parole se folosește BCrypt, pentru refresh token se folosește SHA-256. Asta are sens deoarece refresh token-ul este deja o valoare random cu entropie mare, nu o parolă aleasă de om.

8. Parole BCrypt

Parolele utilizatorilor sunt tratate diferit de refresh tokens. Parola este aleasă de om, deci poate fi slabă sau reutilizată. De aceea se folosește BCryptPasswordEncoder.

Moment	Ce face codul
Register	<code>passwordEncoder.encode(rawPassword)</code> și salvează <code>password_hash</code>
Login	<code>passwordEncoder.matches(rawPassword, user.getPasswordHash())</code>
Baza de date	<code>app_users.password_hash</code> , niciodată parola în clar

BCrypt include salt și este intenționat costisitor. Asta face atacurile brute-force mult mai scumpe dacă hash-urile parolelor ar ajunge la un atacator.

9. SecurityConfig: rute publice, rute protejate, roluri

SecurityConfig definește regula de intrare în API. Aplicația este stateless, CSRF este dezactivat în mod deliberat, iar JwtAuthenticationFilter este pus înaintea UsernamePasswordAuthenticationFilter.

Rută	Regulă
POST /api/auth/register	Public
POST /api/auth/login	Public
POST /api/auth/refresh	Public, dar are nevoie de cookie refresh valid
POST /api/auth/logout	Public, dar revocă refresh token-ul dacă există
GET /api/health	Public
/api/admin/**	ROLE_ADMIN
/api/coach/**	ROLE_COACH
/api/**	Utilizator autentificat
restul	SPA static assets și client routes

CSRF este dezactivat pentru că operațiunile normale nu folosesc cookie ca mecanism de autentificare, ci Authorization header. Cookie-ul sensibil este refresh token-ul, limitat la /api/auth și SameSite=Strict.

10. RBAC: acces pe roluri

RBAC înseamnă Role-Based Access Control. În proiect, rolurile sunt USER, COACH și ADMIN. Rolurile decid accesul la zone mari ale aplicației.

```
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/coach/**").hasRole("COACH")
.requestMatchers("/api/**").authenticated()
```

Interpretare:

- un user neautentificat primește 401 pe endpoint-uri protejate;
- un user autentificat fără rolul necesar primește 403;
- un coach are acces la zona /api/coach/** doar dacă are rol COACH;
- rolul COACH nu îi dă automat acces la toți clienții. Acolo intervine ReBAC.

11. IDOR și ownership checks

IDOR înseamnă Insecure Direct Object Reference. Problema apare când utilizatorul poate schimba un id din URL și accesează resursa altcuiva.

Risc:

```
GET /api/templates/111 -> template-ul meu
GET /api/templates/222 -> template-ul altcuiva
```

Protecție:

backend-ul verifică id-ul resursei împreună cu userId-ul din JWT

În proiect, acest lucru apare în repository-uri owner-scoped și în clase dedicate de access.

Zonă	Mecanism
Template direct	findByIdAndUserIdAndDeletedAtIsNotNull(templateId, userId)
Template day	template_day -> workout_template -> user_id
Template day exercise	template_day_exercise -> template_day -> workout_template -> user_id
Workout session	findByIdAndUserId(sessionId, userId)
Set	workout_set -> session_exercise -> workout_session -> user_id
Gym/equipment/routine	query-uri cu id + user_id + deleted_at is null

Dacă o resursă există, dar aparține altui utilizator, aplicația răspunde ca și cum nu ar exista pentru caller. De aceea se folosesc excepții de tip NotFound și 404, nu 403, pentru multe resurse personale.

12. De ce 404 în loc de 403 la resursele altui user

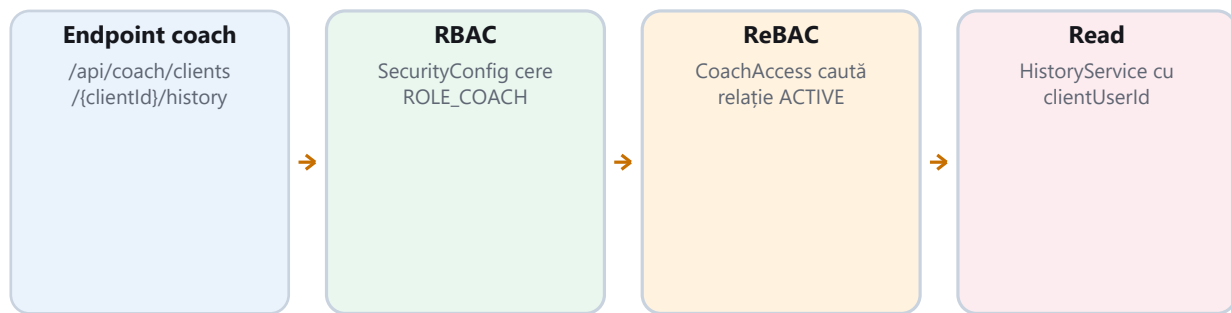
Pentru resurse personale, 403 ar confirma existența resursei. 404 reduce scurgerea de informații. Aceasta este o măsură anti-enumerare.

Situație	Răspuns	Motiv
Nu trimiți JWT pe endpoint protejat	401 Unauthorized	Nu ești autentificat
Ești USER și intri pe /api/coach/**	403 Forbidden	Ești autentificat, dar nu ai rolul necesar
Ceri template-ul altui utilizator	404 Not Found	Nu trebuie confirmată existența resursei
Coach fără relație ACTIVE cere client	404 Not Found	Nu trebuie confirmat că userul există sau are date

Regula de prezentare: 401 este despre lipsa autentificării, 403 este despre lipsa unui rol/permisiuni generale, 404 este folosit intenționat pentru resurse personale sau relații lipsă ca să nu dezvăluie existența lor.

13. ReBAC: relația coach-client

ReBAC înseamnă Relationship-Based Access Control. În coaching, rolul COACH nu este suficient. Coach-ul trebuie să aibă o relație ACTIVE cu clientul.



```

CoachAccess.requireActiveClient(coachUserId, clientId):
  findByCoachUserIdAndClientIdAndStatus(
    coachUserId,
    clientId,
    RelationshipStatus.ACTIVE
  )

```

Unde este folosit:

- CoachViewService.clientHistory - verifică relația, apoi apelează historyService.list(clientUserId);
- CoachViewService.clientAnalytics - verifică relația, apoi analyticsService.overview(clientUserId);
- CoachViewService.clientSession - verifică relația, apoi workoutSessionService.get(clientUserId, sessionId);
- CoachSearchService.searchClientWorkouts - verifică relația, apoi caută istoricul clientului.

Avantajul implementării: după poarta ReBAC, aplicația reutilizează serviciile owner-scoped existente. Coach-ul nu primește query-uri speciale nesigure și nu poate muta/modifica datele clientului prin acele endpoint-uri.

14. Clientul controlează relația cu coach-ul

Relația coach-client are lifecycle: PENDING, ACTIVE, REJECTED, REVOKED. Coach-ul poate invita, dar clientul trebuie să accepte. Clientul poate revoca accesul ulterior.

Ațiune	Cine o face	Verificare de securitate
Invite client	Coach	Endpoint /api/coach/** cere ROLE_COACH
Accept invite	Client	RelationshipService caută relația după id + clientId din JWT
Reject invite	Client	La fel, doar clientul relației poate respinge
Revoke coach	Client	Doar relația activă a clientului curent poate fi revocată
Read client history	Coach	ROLE_COACH + relație ACTIVE

Asta este important pentru prezentare: userul client păstrează controlul asupra accesului la datele sale.

15. CORS, CSRF și SPA

Frontend-ul poate rula ca SPA servit de backend sau, în development, pe Vite la localhost:5173. CORS permite originea configurată și permite credentials pentru ca browserul să poată stoca/trimită cookie-ul HttpOnly.

- Allowed methods: GET, POST, PUT, DELETE, PATCH, OPTIONS.
- Allowed headers: Authorization, Content-Type, Accept.

- Allow credentials: true, necesar pentru refresh cookie în development.
- CSRF dezactivat deoarece operațiile normale folosesc Authorization header, nu cookie de sesiune.
- Refresh cookie are SameSite=Strict și Path=/api/auth, deci este restrâns la fluxul de autentificare.

Dacă te întreabă de CSRF: modelul nu folosește cookie ca autentificare pentru toate cererile API. Cookie-ul este doar pentru refresh/logout și este restricționat.

16. Cum se vede securitatea în baza de date

Tabel	Câmpuri relevante	Rol de securitate
app_users	email, password_hash, role	Identitate, roluri, parole hash-uite
refresh_tokens	user_id, token_hash, expires_at, revoked_at	Refresh token server-side, revocabil și rotit
coach_client_relationships	coach_user_id, client_user_id, status	Model ReBAC pentru coaching
workout_templates	user_id, visibility, deleted_at	Ownership pentru programe private/publiche
workout_sessions	user_id, status	Istoricul este owner-scoped

Baza de date contribuie și prin constrângeri: roluri valide, statusuri valide, relații coach-client fără self-coaching, indecși unici pe email și token_hash, plus chei străine care șterg/revocă datele dependente când userul dispare.

17. Limitări reale și răspunsuri sincere

Este bine să poți spune și ce nu acoperă sistemul. Asta arată că înțelegi securitatea, nu că vinzi ceva magic.

Întrebare posibilă	Răspuns bun
Dacă cineva fură access token-ul?	Poate acționa până expiră, dar fereastra este scurtă. Refresh token-ul este protejat separat.
Dacă cineva fură refresh token-ul?	Rotația și revoked_at reduc reutilizarea, iar token-ul nu este accesibil JavaScript prin HttpOnly.
De ce nu folosești sesiuni server-side?	JWT stateless simplifică API-ul și scalarea, iar refresh token-ul rămâne revocabil server-side.
De ce uneori 404, nu 403?	Pentru resurse personale, 404 evită confirmarea existenței resursei altui user.
Coach-ul poate modifica datele clientului?	Nu prin endpoint-urile implementate de coach. Sunt read-only, iar accesul cere relație ACTIVE.
Ce nu ai implementat?	Nu am rate limiting/2FA. Pentru producție acestea ar fi extensii naturale.

18. Explicația scurtă de spus la prezentare

Securitatea aplicației este construită pe mai multe niveluri. Autentificarea se face cu access token JWT de durată scurtă și refresh token opac. Refresh token-ul este pus într-un cookie HttpOnly, SameSite=Strict, limitat la /api/auth, iar în baza de date este păstrat doar ca SHA-256 fingerprint. La refresh, token-ul vechi este revocat și se emite o pereche nouă. Parolele sunt stocate cu BCrypt.

După autentificare, backend-ul nu are încredere în userId-uri trimise de frontend. Utilizatorul curent este luat din JWT și fiecare resursă este încărcată cu verificări de ownership. Pentru resursele altui user, sistemul

răspunde cu 404 pentru a evita IDOR și enumerarea de identificatori. Pentru roluri folosesc RBAC, iar pentru coaching folosesc ReBAC: un coach poate vedea datele unui client doar dacă există o relație ACTIVE între ei.

19. Checklist de memorat

- JWT = access token scurt, trimis în Authorization header.
- Refresh token = opac, random, în cookie HttpOnly, rotit la refresh.
- SHA-256 pentru refresh token în DB, BCrypt pentru parole.
- Frontend-ul nu decide userId-ul. Backend-ul ia userul din JWT.
- IDOR se previne prin owner-scoped queries și access helpers.
- 404 pentru resursele altui user, ca să nu le confirmi existența.
- RBAC = roluri pe zone mari: ADMIN, COACH, USER.
- ReBAC = coach-ul are nevoie de relație ACTIVE cu clientul.
- Endpoint-urile de coach sunt read-only pentru datele clientului.
- SecurityConfig este stateless și pune JwtAuthenticationFilter în lanțul Spring Security.

Fraza-cheie: autentificarea spune cine e userul, dar fiecare operație importantă verifică separat dreptul concret de acces la resursa cerută.